

CODING & PROGRAMMING FUNDAMENTALS

PYTHON BEGINNER PATH — COMPLETE VISUAL STUDY MANUAL

Paid Enrollee Course Material

Course Code: EDA-COD-171

Level: Beginner

Duration: 8 Weeks (18 Lessons)

Certificate: Professional Certificate in Coding & Programming Fundamentals

Prerequisite: Computer Appreciation / Digital Literacy

Learning Format: Classroom + Guided Coding Practicals + Final Project

COURSE OVERVIEW

Coding is the process of writing instructions that a computer can execute. Programming goes further by combining code, logic, data and problem-solving to create useful software. This course introduces these ideas through Python, a readable programming language suitable for beginners.

The course is written for learners, not professional developers. Every lesson explains the concept first, shows a small example, and then gives a guided practical that can be completed in a classroom or beginner coding environment.

LEARNING OBJECTIVES

- Explain coding, programming, algorithms and programs.
- Plan solutions with algorithms and flowcharts.
- Use Python variables and basic data types.
- Accept input and display output.
- Perform arithmetic and comparison operations.
- Use IF/ELIF/ELSE decisions.
- Use FOR and WHILE loops.
- Create reusable functions.
- Work with strings, lists and dictionaries.
- Handle simple errors and debug code systematically.
- Read and write simple text/CSV-style data.
- Understand modules and basic object-oriented thinking.
- Build, test and document a complete beginner project.

COURSE OUTLINE

Lesson	Topic
1	Introduction to Coding and Programming
2	Algorithms, Pseudocode and Flowcharts
3	Python Environment and First Program
4	Variables and Data Types
5	Input, Output and Type Conversion
6	Operators and Calculations
7	Decision Making with IF
8	FOR Loops
9	WHILE Loops
10	Functions
11	Strings
12	Lists, Tuples and Dictionaries
13	Debugging and Error Handling
14	Files and Simple Data
15	Modules and Reusable Code
16	Introduction to Object-Oriented Programming
17	Planning and Testing a Software Project
18	Final Python Project

LESSON 1: INTRODUCTION TO CODING AND PROGRAMMING

A computer follows instructions. Coding expresses those instructions in a programming language, while programming combines instructions with problem-solving and software design.

How a Program Works

Input → Processing → Output

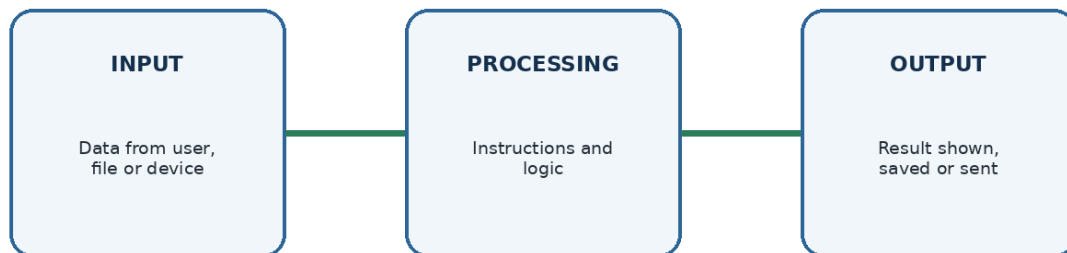


Figure 1. The input-processing-output model.

Key Terms

- Code - instructions written in a programming language.
- Program - a set of instructions that performs a task.
- Programming language - a structured way to communicate instructions to a computer.
- Programmer - a person who designs and writes programs.

Input, Processing and Output

Many programs accept input, process it according to rules, and produce output.

Examples of Programs

Calculators, school result systems, games, websites, banking applications and mobile apps all use programmed logic.

Guided Practical

1. List five programs you use.
2. For each, identify one possible input and output.
3. Write three problems that a simple program could help solve.

Review Questions

1. What is coding?
2. What is programming?

3. What is a program?

4. Explain input, processing and output.

Assignment

Create a one-page poster explaining how a program works.

LESSON 2: ALGORITHMS, PSEUDOCODE AND FLOWCHARTS

Before writing code, programmers can describe the solution as a sequence of logical steps.

Algorithm and Flowchart

Plan the solution before writing code



Figure 2. An algorithm and equivalent flowchart.

Algorithm

An algorithm is a clear sequence of steps for solving a problem.

Pseudocode

Pseudocode describes program logic in plain structured language without requiring exact programming syntax.

Flowchart

A flowchart uses shapes and arrows to show the sequence of actions and decisions.

Good Planning

A good plan identifies inputs, processing, decisions and outputs before coding begins.

Guided Practical

1. Write an algorithm for adding two numbers.
2. Write pseudocode for deciding whether a score is Pass or Review.
3. Draw a flowchart for the addition algorithm.

Review Questions

1. What is an algorithm?
2. What is pseudocode?

3. What is a flowchart?
4. Why plan before coding?

Assignment

Plan an algorithm for calculating the average of three scores.

LESSON 3: PYTHON ENVIRONMENT AND FIRST PROGRAM

Python code can be written in an approved beginner environment and executed to produce results.

How a Program Works

Input → Processing → Output

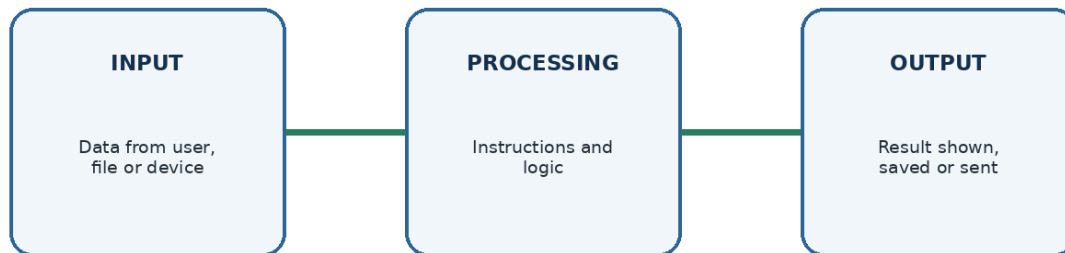


Figure 1. Programs transform instructions into visible output.

Python Syntax

Syntax means the rules for writing valid Python code.

First Output

The `print()` function displays information.

Comments

A comment begins with `#` and can explain code for human readers.

Indentation

Python uses indentation to show blocks of related code. Consistent indentation matters.

Code Example

```
print("Welcome to Ethan Digital Academy")
```

Guided Practical

1. Open the instructor-approved Python environment.
2. Type and run the first program below.
3. Change the message and run it again.
4. Add a comment explaining the program.

Review Questions

1. What does `print()` do?
2. What is syntax?
3. What is a comment?
4. Why does indentation matter?

Assignment

Write a program that displays your academy, course and a learning goal.

LESSON 4: VARIABLES AND DATA TYPES

A variable gives a meaningful name to a value so the program can use and change that value.

Variables and Data Types

Variables give names to values



Figure 3. Variables and common Python data types.

Naming Variables

Use clear names such as `student_name` or `course_fee`. Avoid spaces in variable names.

Basic Data Types

- `str` - text
- `int` - whole numbers
- `float` - decimal numbers
- `bool` - True or False

Assignment

The equals sign assigns a value to a variable.

Checking Types

Python's `type()` function can show the type of a value during learning and debugging.

Code Example

```
student_name = "Ada"
age = 16
course_fee = 65000
active = True
```

Guided Practical

1. Create variables for a learner name, age, course fee and active status.

2. Display each variable.
3. Use `type()` to inspect the values.
4. Change one value and run the program again.

Review Questions

1. What is a variable?
2. Name four basic data types.
3. What does `=` do in an assignment?
4. Why are meaningful variable names useful?

Assignment

Create a learner-profile program using four data types.

LESSON 5: INPUT, OUTPUT AND TYPE CONVERSION

Interactive programs can receive information from a user and use it in calculations or messages.

Variables and Data Types

Variables give names to values



Figure 3. Data types determine how values behave.

input()

The input() function receives text entered by the user.

Type Conversion

Use int() or float() when numeric input is required for calculations.

Formatted Output

f-strings can combine text and variable values clearly.

Validation Awareness

Real programs should check unexpected input. Beginner exercises will start with expected classroom values.

Code Example

```
name = input("Enter name: ")
score = float(input("Enter score: "))
print(f"{name}: {score}")
```

Guided Practical

1. Ask the user for a name.
2. Ask for two numbers and convert them to float.
3. Add the numbers.

4. Display the result with an f-string.

Review Questions

1. What does `input()` return by default?
2. Why is type conversion needed?
3. What does `float()` do?
4. What is an f-string used for?

Assignment

Create a simple course-fee enquiry program.

LESSON 6: OPERATORS AND CALCULATIONS

Operators allow programs to calculate values and compare information.

How a Program Works

Input → Processing → Output

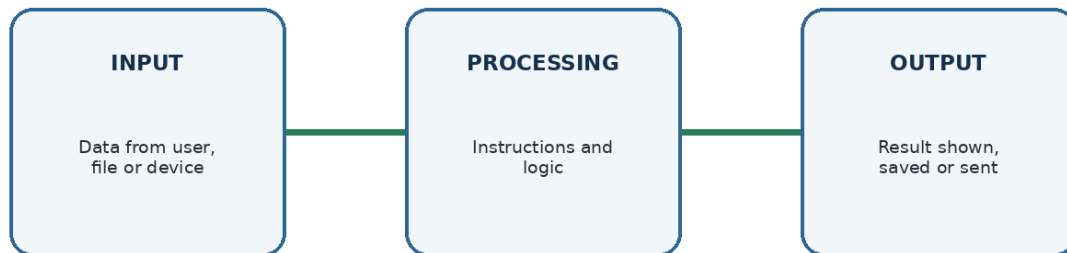


Figure 1. Programs process input using instructions and calculations.

Arithmetic Operators

- + addition
- - subtraction
- * multiplication
- / division
- % remainder
- ** exponentiation

Comparison Operators

- == equal to
- != not equal to
- > greater than
- < less than
- >= greater than or equal to
- <= less than or equal to

Order of Operations

Parentheses can make calculations clearer and control evaluation order.

Practical Calculation

A program can calculate totals, averages, discounts and simple business figures.

Guided Practical

1. Create variables for quantity and unit_price.

2. Calculate total = quantity * unit_price.
3. Calculate the average of three scores.
4. Test whether the average is at least 50.

Review Questions

1. What does * do?
2. What does == mean?
3. What does % return?
4. Why use parentheses?

Assignment

Build a simple shopping-total calculator.

LESSON 7: DECISION MAKING WITH IF

Conditional statements let a program choose different actions depending on whether a condition is true or false.

Decision Making with IF

Programs can choose actions based on conditions

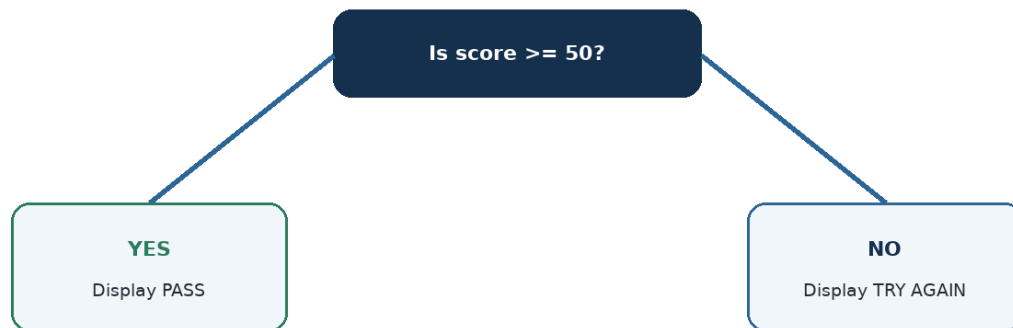


Figure 4. A decision flow using IF.

if

Runs a block when a condition is true.

else

Runs an alternative block when the IF condition is false.

elif

Adds another condition between IF and ELSE.

Logical Operators

- and - both conditions must be true
- or - at least one condition must be true
- not - reverses a Boolean condition

Code Example

```
score = 68
if score >= 75:
    print("Distinction")
elif score >= 50:
    print("Pass")
else:
    print("Review")
```

Guided Practical

1. Enter a score.
2. Use IF to display Pass when score is at least 50.
3. Add ELIF for Distinction at 75 or above.
4. Add ELSE for Review.
5. Test values around the boundaries.

Review Questions

1. What is a condition?
2. What does ELSE do?
3. When is ELIF useful?
4. Differentiate AND and OR.

Assignment

Create an age-appropriate course-category selector using fictional values.

LESSON 8: FOR LOOPS

A FOR loop repeats instructions over a sequence or a known range.

Loops

Repeat a task without rewriting the same instruction

WITHOUT LOOP

```
print("Welcome")  
print("Welcome")  
print("Welcome")  
print("Welcome")
```

WITH LOOP

```
for i in range(4):  
    print("Welcome")
```

Figure 5. A loop replaces repeated instructions.

range()

range() can generate a sequence of numbers for controlled repetition.

Loop Variable

The loop variable takes each value in the sequence in turn.

Accumulation

A loop can add values into a running total.

Avoid Unnecessary Repetition

Loops make code shorter and easier to maintain when the same pattern repeats.

Code Example

```
for number in range(1, 6):  
    print(number)
```

Guided Practical

1. Print the numbers 1 to 5.
2. Print Welcome four times.
3. Calculate the total of numbers 1 to 10 using a loop.
4. Loop through a short list of course names.

Review Questions

1. What is a FOR loop?
2. What does range() do?
3. What is a loop variable?
4. Why are loops useful?

Assignment

Create a multiplication-table program for one chosen number.

LESSON 9: WHILE LOOPS

A WHILE loop continues while a condition remains true.

Loops

Repeat a task without rewriting the same instruction

WITHOUT LOOP

```
print("Welcome")
print("Welcome")
print("Welcome")
print("Welcome")
```

WITH LOOP

```
for i in range(4):
    print("Welcome")
```

Figure 5. Loops repeat instructions efficiently.

Condition-Controlled Repetition

WHILE is useful when the number of repetitions is not known in advance.

Updating the Condition

The program must eventually change something that can make the condition false.

Infinite Loops

A loop that never reaches a stopping condition can continue indefinitely. Design a clear exit condition.

Counters

A counter can track how many times a loop has run.

Code Example

```
counter = 1
while counter <= 5:
    print(counter)
    counter += 1
```

Guided Practical

1. Set counter = 1.
2. Use WHILE to display 1 through 5.
3. Increase the counter each time.

4. Change the stopping value and test again.

Review Questions

1. What is a WHILE loop?
2. What is an infinite loop?
3. Why must the condition change?
4. What is a counter?

Assignment

Create a countdown program from 10 to 1.

LESSON 10: FUNCTIONS

Functions package related instructions into reusable named blocks.

Functions

Package reusable instructions into named blocks

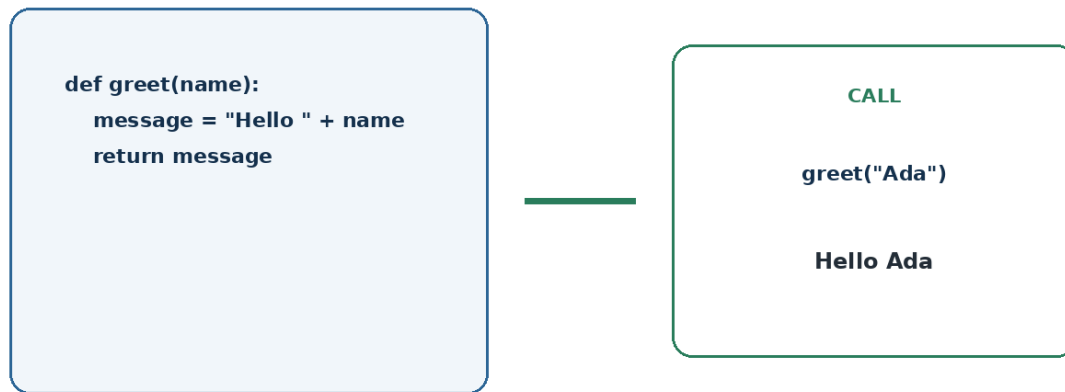


Figure 6. A reusable function with a parameter and return value.

Defining a Function

Use `def` followed by the function name and parentheses.

Parameters

Parameters receive values supplied when the function is called.

Return Values

`return` sends a result back to the caller.

Benefits

Functions improve reuse, readability, testing and organisation.

Code Example

```
def add(a, b):  
    return a + b
```

```
result = add(4, 6)  
print(result)
```

Guided Practical

1. Create `greet(name)`.
2. Call the function with two different fictional names.

3. Create `add(a, b)` that returns `a+b`.
4. Store and display the returned result.

Review Questions

1. What is a function?
2. What is a parameter?
3. What does `return` do?
4. Give two benefits of functions.

Assignment

Create three small reusable functions for a result program.

LESSON 11: STRINGS

Strings store text and provide useful operations for searching, cleaning and formatting text.

Lists and Dictionaries

Store collections of related information

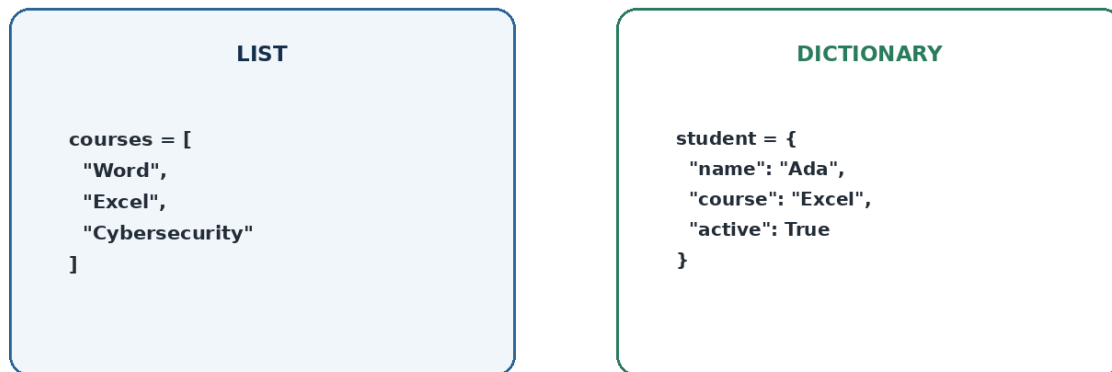


Figure 7. Collections and text are common program data.

Indexing

Characters in a string can be accessed by position.

Useful Methods

- `lower()`
- `upper()`
- `strip()`
- `replace()`
- `split()`

Length

`len()` returns the number of characters.

Combining Text

Strings can be joined with `+` or formatted with f-strings.

Guided Practical

1. Create a `course_name` string.
2. Display it in upper and lower case.
3. Use `strip()` on text with extra spaces.
4. Use `len()` to count characters.

5. Split a simple comma-separated string.

Review Questions

1. What is a string?
2. What does `len()` do?
3. What does `strip()` do?
4. What is `split()` used for?

Assignment

Create a name-cleaning program.

LESSON 12: LISTS, TUPLES AND DICTIONARIES

Collections allow programs to store several related values.

Lists and Dictionaries

Store collections of related information

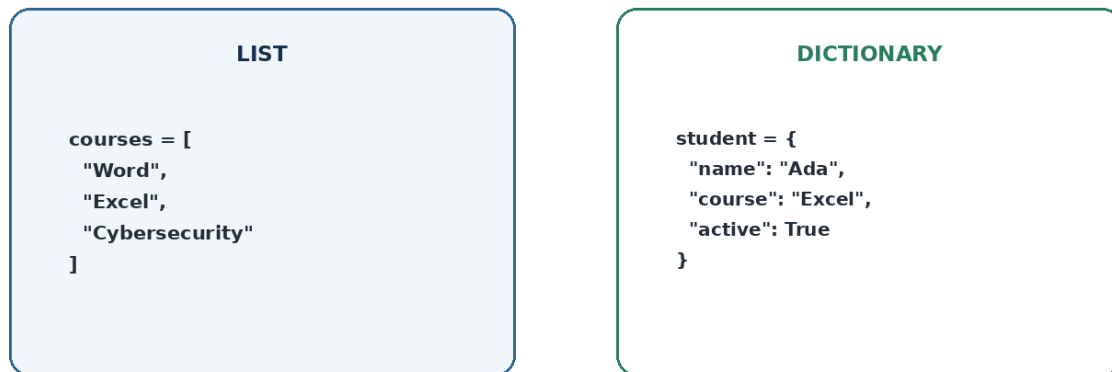


Figure 7. Lists and dictionaries.

Lists

Lists are ordered and can be changed.

Tuples

Tuples are ordered collections commonly used when values should remain fixed in a simple program.

Dictionaries

Dictionaries store key-value pairs such as name → Ada.

Looping Collections

FOR loops can process each item in a collection.

Code Example

```
courses = ["Word", "Excel", "Cybersecurity"]  
student = {"name": "Ada", "course": "Excel"}
```

Guided Practical

1. Create a list of five courses.
2. Add one course and remove one.
3. Loop through the list.
4. Create a dictionary for one fictional learner.

5. Display the learner's course using its key.

Review Questions

1. What is a list?
2. What is a tuple?
3. What is a dictionary?
4. What is a key-value pair?

Assignment

Create a small course catalogue using a list of dictionaries.

LESSON 13: DEBUGGING AND ERROR HANDLING

Debugging is the systematic process of finding and correcting problems in a program.

Debugging Workflow

Read → Reproduce → Inspect → Fix → Test



Figure 8. A structured debugging workflow.

Syntax Errors

The code breaks Python's writing rules.

Runtime Errors

The program starts but encounters a problem while running.

Logic Errors

The program runs but produces an incorrect result.

Debugging Method

Read the message, reproduce the problem, inspect the relevant code, make a focused change and test again.

Basic Exception Handling

try/except can handle expected errors such as invalid numeric input, but it should not be used to hide every problem.

Code Example

```
try:
    score = float(input("Enter score: "))
except ValueError:
    print("Please enter a number.")
```

Guided Practical

1. Run a teacher-provided program with a harmless syntax mistake.
2. Read the error message and correct it.
3. Create a simple numeric input example with try/except.
4. Test expected and unexpected input.

Review Questions

1. Name three error categories.
2. What is a logic error?
3. What is debugging?
4. What does try/except help manage?

Assignment

Write a debugging checklist for beginners.

LESSON 14: FILES AND SIMPLE DATA

Programs can save information to files and read it again later.

Files and Data

Programs can read and save structured information

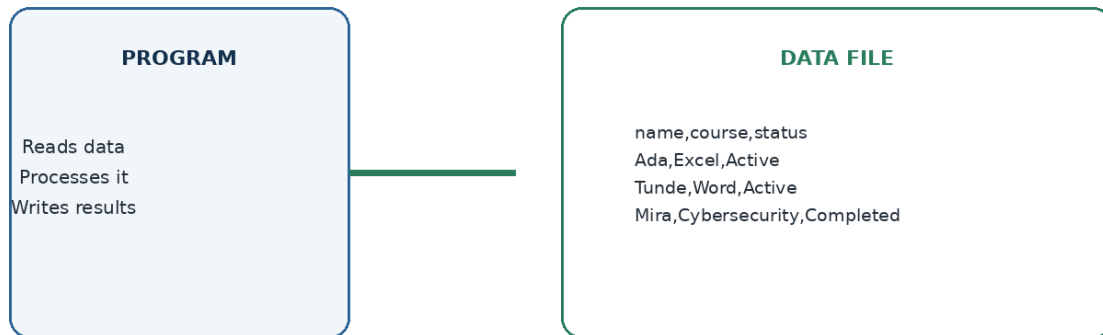


Figure 9. A program reading and writing simple data.

Text Files

Text files store human-readable characters.

CSV-Style Data

Comma-separated rows can represent simple tables.

Reading and Writing

Python can open a file for reading or writing. Close files properly; using `with open(...)` handles this automatically.

File Safety

Use only your own practice files. Do not overwrite important files during exercises.

Code Example

```
with open("practice.txt", "w") as file:  
    file.write("Microsoft Excel\n")
```

Guided Practical

1. Create a practice text file.
2. Write three course names to it.
3. Read the file and display the contents.

4. Create a simple CSV-style file with name,course,status.

Review Questions

1. What is a text file?
2. What is CSV-style data?
3. Why is with open useful?
4. What file-safety rule applies in class?

Assignment

Create a program that saves a simple learner report to a new practice file.

LESSON 15: MODULES AND REUSABLE CODE

Modules help organise code and provide reusable functions.

Functions

Package reusable instructions into named blocks

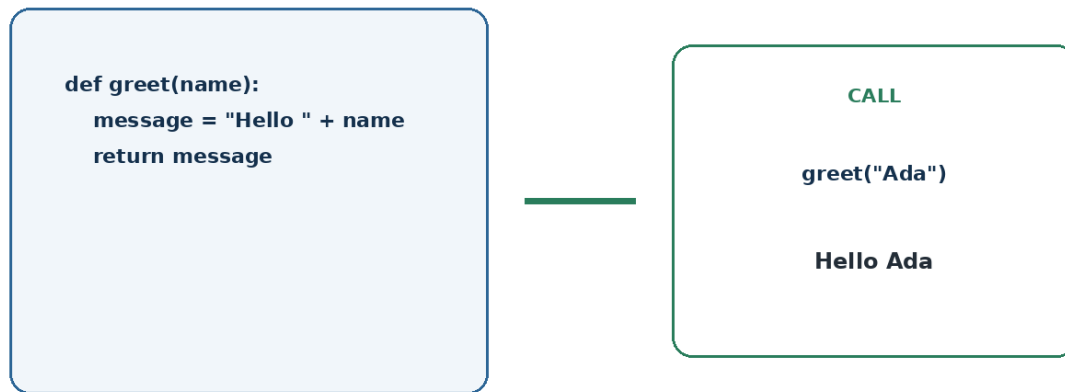


Figure 6. Functions are building blocks for reusable modules.

Importing

import makes approved module features available to a program.

Standard Library

Python includes many built-in modules for common tasks.

Own Modules

As projects grow, related functions can be placed in separate Python files.

Responsible Use

Understand what a library does before using it and use trusted sources for external packages.

Guided Practical

1. Import the math module.
2. Use `math.sqrt()` on a positive classroom value.
3. Create a small file with one reusable function if your environment supports multiple files.
4. Import and call the function.

Review Questions

1. What is a module?
2. What does import do?

3. What is the standard library?
4. Why should external packages come from trusted sources?

Assignment

Prepare a one-page explanation of reusable code.

LESSON 16: INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING

Object-oriented programming groups related data and behaviour into classes and objects.

Object-Oriented Thinking

Group data and behaviour around an object

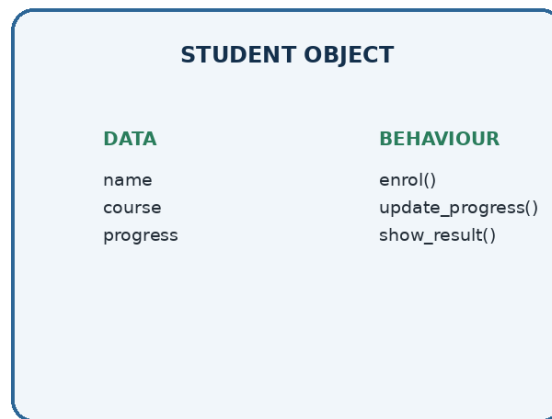


Figure 10. Data and behaviour grouped in an object.

Class

A class is a blueprint describing attributes and behaviours.

Object

An object is an instance created from a class.

Attributes

Attributes store data about the object.

Methods

Methods are functions associated with the object.

Beginner Perspective

OOP is introduced here conceptually; mastery comes with larger projects and further study.

Code Example

```
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course
```

Guided Practical

1. Define a simple Student class with name and course attributes.

2. Create one fictional Student object.
3. Add a method that displays a summary.
4. Create a second object and compare the values.

Review Questions

1. What is a class?
2. What is an object?
3. What is an attribute?
4. What is a method?

Assignment

Draw an object model for Student or Course.

LESSON 17: PLANNING AND TESTING A SOFTWARE PROJECT

A successful beginner project starts with a clear problem, small features and planned tests.

Software Project Workflow

Build small, testable pieces



Figure 11. A simple software-project workflow.

Requirements

State what the program must do and what is outside scope.

Decomposition

Break a large problem into smaller tasks or functions.

Test Cases

Test normal values, boundary values and simple unexpected input.

Documentation

Explain the purpose, inputs, outputs and how to run the program.

Version Discipline

Save working copies as the project improves so mistakes can be reversed.

Guided Practical

1. Write requirements for a student result manager.
2. Break the program into five functions/tasks.
3. Create six test cases.
4. Write a short user guide.

Review Questions

1. What is a requirement?
2. What is decomposition?
3. What is a test case?
4. Why is documentation important?

Assignment

Submit a project plan before coding the final project.

LESSON 18: FINAL PYTHON PROJECT

The final project combines input, variables, calculations, decisions, loops, functions, collections and simple file output.

Final Project Architecture

A simple student result manager

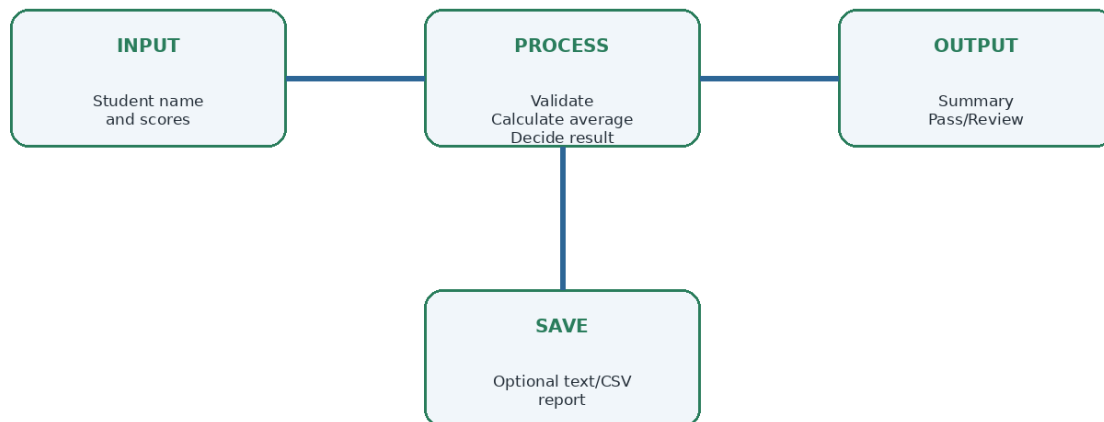


Figure 12. Architecture of the final Student Result Manager.

Project: Student Result Manager

Create a program that accepts a fictional learner name and several scores, validates the expected numeric range, calculates total and average, determines a result category, displays a clear summary and optionally saves a report to a new practice file.

Required Skills

- Input/output
- Variables and data types
- Arithmetic
- IF/ELIF/ELSE
- Loop
- Function
- List or dictionary
- Basic error handling
- Readable output
- Testing and documentation

Quality Standard

Use meaningful names, clear prompts, sensible functions and comments where they add value.

Guided Practical

1. Plan the input and output.

2. Write the calculation function.
3. Write the result-category function.
4. Add a loop for scores.
5. Add safe input handling.
6. Display the summary.
7. Test at least six cases.
8. Document and submit the program.

Review Questions

1. Which concepts does the project combine?
2. Why should score boundaries be tested?
3. Why use functions?
4. What should the user guide explain?

Assignment

Submit the complete program, test table and short user guide.

REAL-LIFE CASE STUDY - ETHAN DIGITAL ACADEMY

Ethan Digital Academy wants a simple classroom program for recording practice assessment results. The learner first plans the algorithm, then creates variables for names and scores, uses a loop to collect several scores, calculates the average with a function, uses IF/ELIF/ELSE to classify the result and displays a readable report. The program is tested with normal and boundary values before a practice report is saved. This small project demonstrates how programming combines logic, data, testing and communication.

FINAL PRACTICAL PROJECT - 100 MARKS

1. Write the project requirements and algorithm.
2. Create a flowchart or pseudocode.
3. Accept a fictional learner name and multiple scores.
4. Use a list to store scores.
5. Use a loop to process the scores.
6. Create functions for calculation and result classification.
7. Use IF/ELIF/ELSE for the result category.
8. Handle simple invalid numeric input.
9. Display a professional summary and optionally save a practice report.
10. Submit six test cases and a short user guide.

QUIZ - 30 QUESTIONS

1. What is coding?
2. What is programming?
3. What is an algorithm?
4. What is pseudocode?
5. What is a flowchart?
6. What does `print()` do?
7. What is a variable?
8. Name four Python data types.
9. What does `input()` do?
10. Why is type conversion needed?
11. What does `*` mean?
12. What does `==` mean?

13. What is a condition?
14. What does ELIF do?
15. What is a FOR loop?
16. What is a WHILE loop?
17. What is an infinite loop?
18. What is a function?
19. What is a parameter?
20. What does return do?
21. What is a string?
22. What is a list?
23. What is a dictionary?
24. What is a syntax error?
25. What is a logic error?
26. What is debugging?
27. What is a text file?
28. What is a module?
29. What is a class?
30. What is a test case?

FINAL EXAMINATION - 100 MARKS

Section A - Objective Questions (20 Marks)

Twenty questions covering programming concepts, Python syntax, data types, decisions, loops, functions and collections.

Section B - Short Answer and Code Reading (30 Marks)

Explain concepts, trace short beginner programs and identify simple errors.

Section C - Practical Programming Assessment (50 Marks)

Build a small Python program from a supplied classroom problem using input, calculations, decisions, loops, functions and testing.

COURSE COMPLETION OUTCOME

Graduates of EDA-COD-171 Coding & Programming Fundamentals will understand core programming logic and be able to create, test and explain small Python programs. They will be prepared for Python for Beginners, JavaScript Programming, Software Development Basics, API Fundamentals and App Development Fundamentals.